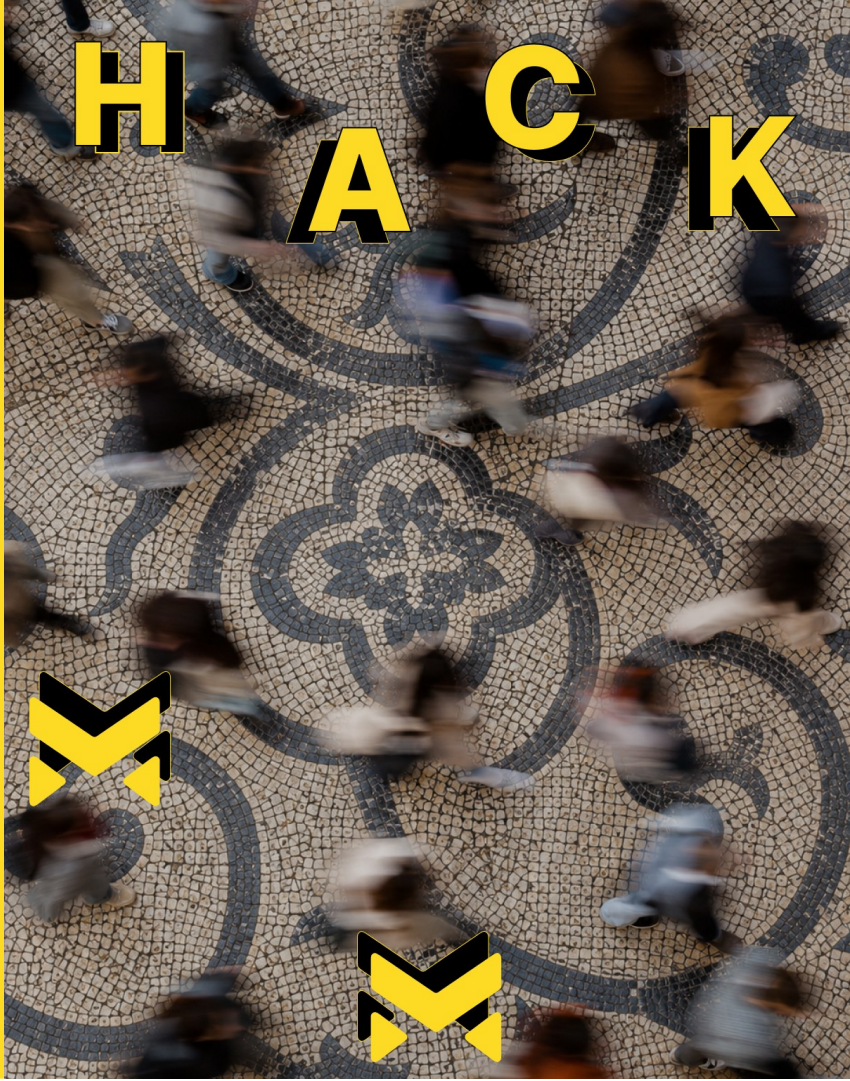
Hack Meridian / 2026 LISBON

BUILDING AT HACK MERIDIAN

STELLAR SMART ACCOUNTS

What a C-address gets you, and how to build an app on one

Stellar





AN ACCOUNT WITH RULES

A G-address is a key. A C-address is a contract that decides what each signature may do.

CLASSIC ACCOUNTS

G-ADDRESS: A KEY

Signers are ed25519 keys with weights and thresholds. The protocol fixes the rules; you set the numbers.

You can’t say “only this contract”, “only until Friday” or “at most 2 XLM a day”.

So an app asks you to sign every action, or holds a key that can do everything.

Multisig, yes. Scoped permissions, no.



C-ADDRESS: THE ACCOUNT RUNS CODE

Rust

```rust
pub fn check_in(env: Env, who: Address) -> u32 {
    who.require_auth(); // a C-address runs its __check_auth here
    let count = Self::count(env.clone(), who.clone()) + 1;
    env.storage().persistent().set(&who, &count);
    count
}
```

Same code either way: the host checks a G-address's signatures, and asks a C-address.

WHAT YOU CAN DO WITH RULES

PASSKEYS

Sign in with Face ID or a fingerprint. A verifier contract checks the P-256 signature.

SCOPED KEYS

Give an app a key for one contract and a few functions, until an expiry.

SPENDING CAPS

Attach a limit, like 2 XLM a day. The account enforces it on every transfer.

RECOVERY

Guardians or a recovery kit, set up before you need them. It’s opt-in.

SPONSORED START

Someone else pays to create the account and submit for it. Nido does on testnet.

ONE ADDRESS

Add or remove devices and app keys by changing rules. The address stays.

THE TRADE-OFFS

WHAT NIDO'S RELAYER PAYS

Account: about 1.4 XLM
First grant: about 4.5 XLM
Capped grant: about 2.7 XLM

One 100 XLM/day pot for all of testnet

COMPATIBILITY

No classic operations; no Horizon account record

COST

Rent for the account and for every rule

MATURITY

Testnet only, not audited, forked dependencies

PERCH: A RULE YOU CAN READ

JSON

```
{
  "name": "kudos-xlm",
  "scope": { "type": "contract", "address": "CDLZ…CYSC" },
  "principals": { "type": "all", "signers": ["session"] },
  "functions": ["transfer"],
  "cap": { "limit": "20000000", "period-ledgers": 17280 },
  "not-after-ledger": 5342393
}
```

Perch compiles the account's document on chain; the account enforces it in __check_auth.

Hack Meridian / 2026 LISBON

THE WALLET

NIDO

A passkey wallet in which every account is a Perch account.

One passkey creates a C-address with 5 test XLM; Nido’s relayer pays.

Your app finds it in Stellar Wallets Kit.

test.nido.fyi · testnet



NIDO'S POPUP CAN'T SIGN FOR YOUR CONTRACT

auth trace

```
game.guess(3, CAXS…)                          // the app's call
→ CAXS…::__check_auth(signatures, contexts)
    contexts: guess, xlm.transfer             // two calls
    context_rule_ids: [0]                     // one rule: "admin"
→ Error(Contract, #3014) ContextRuleIdsLengthMismatch
```

A call into one contract fails with #3002 instead. Either way the fix is a session.



A TEST RUN

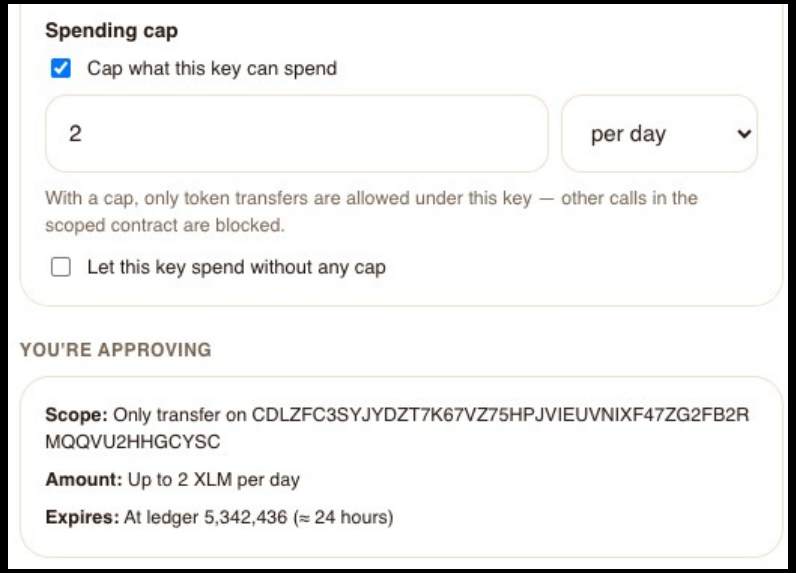
AN AGENT BUILT KUDOS JAR

We gave a coding agent the template and one prompt: build a small smart account app, live on testnet.

It built Kudos Jar: tip a team in XLM, up to 2 XLM a day, with a public tally.

It ran with a real Nido account and a virtual passkey.

APPROVING THE RULE IN NIDO



Nido's grant page, before the passkey

SCOPE

Only transfer, only on the XLM token

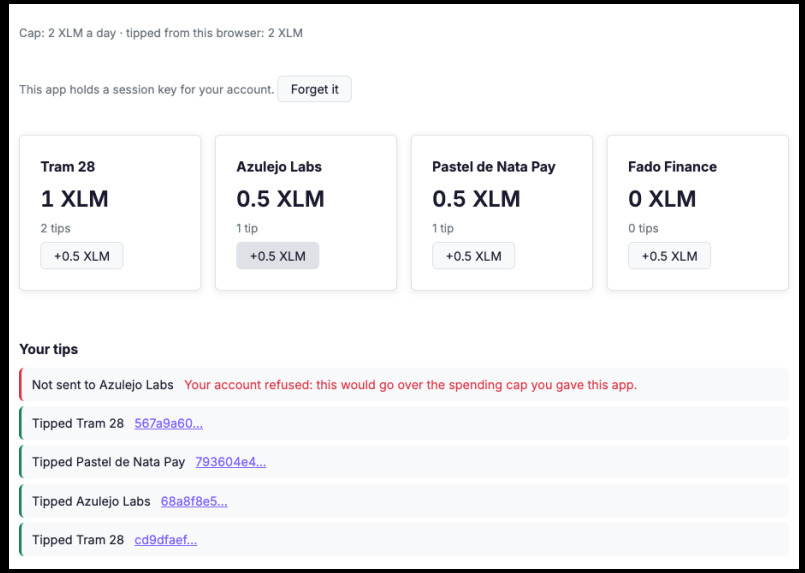
CAP

Up to 2 XLM a day

EXPIRY

About 24 hours, then it stops

FOUR TIPS, THEN A REFUSAL



Live testnet, 22 Oct 2026

NO PROMPTS

Four tips, each confirmed on chain

THE CAP

A fifth would pass 2 XLM for the day

WHO SAID NO

The account, with error #3221

WHEN A GRANT DOESN'T CONFIRM

CHECK THE CHAIN BEFORE YOU RETRY

Go back to the app, press Check again: it reads the account's rules on chain. Nothing there? Ask Nido again.

A timeout can follow a success

429 · BUDGET USED UP

Nothing was sent. Wait, or pay your own fee

503 · RELAYER BUSY

Nothing was sent. Retry in a moment

504 · TIMED OUT

It may have landed. Check, then retry

PATTERNS THAT WORKED

Six AI-driven teams, two hours each on testnet. All six prototypes worked.

A CAPPED PAYMENT

Tips or purchases: one rule on your contract, one on the token with a cap.

NO MONEY MOVES

Tickets, a poll, a guestbook: one rule on your contract and no cap.

SHARED FUNDS

A contract holds the pot and pays out when 2 of 3 members approve.

LIMITS THEY CAN'T UNDO

A cap limits a key, not the owner. Put allowances and time locks in a contract.

THE PART YOU WRITE

TypeScript

```typescript
// One grant per contract in the call's env.auths()
const grants: Grant[] = [
  { target: myContract.options.contractId, functions: ["my_fn"],
    duration: "24h", label: "my-app" },
]
// Inside <SessionGate grants={grants}>, on click:
const { result, hash } = await submitCall(address, (publicKey) =>
  myContract.my_fn({ user: address }, { publicKey }))
```

Keep require_auth first in your contract function. submitCall signs with whatever the user connected.

QUICK START

START FROM THE TEMPLATE

github.com/nidohq/scaffold-template

Its quick start, template/NIDO.md, takes a fresh copy to a deployed contract and a running app.

AGENTS.md has the same rules, written for coding agents.

Testnet only. Throwaway keys only.



KEEP BUILDING

Hack Meridian / 2026 LISBON

YOUR FIRST SMART ACCOUNT APP

Find the one thing your app does
for a user, and the rule that limits it.

test.nido.fyi · testnet
github.com/nidohq/scaffold-template
github.com/stellar-registry/perch





BACKUP SLIDES

Perch in depth, the rest of the walkthrough, and what broke



POLICY DOCUMENTS

ONE DOCUMENT, ONE HASH

A policy is signers, rules and optional recovery, as canonical JSON. The account stores its hash.

Anyone can fetch the document, hash it, and compare it with the chain.

apply_doc is the only way to change an account's authorization.

FROM DOCUMENT TO ENFORCEMENT

THE COMPILER IS A CONTRACT

The account sends its document to Perch's doc compiler, so compiling happens on chain.

RULES BECOME CONTEXT RULES

Each rule lowers onto an OpenZeppelin context rule; a cap adds the spending-limit policy.

CONSTRAINTS ARE PROGRAMS

Rules with constraints get a small program for Perch's interpreter.

EVERY CALL IS CHECKED

The named rule's scope, expiry, signers and every policy must pass. Anything undecodable is denied.

VERIFIERS

ED25519

Classic keys, checked by a verifier contract.

WEBAUTHN

Passkeys on P-256. Nido’s admin key is one.

DELEGATED

Any G or C address, authenticated by the host through CAP-71. The demo’s session key is one.

NOT BUILT YET: ML-DSA

A post-quantum scheme, on Perch’s roadmap as one more verifier contract.

CONTENT-ADDRESSED PUBLISHING

CONSTRUCTORLESS

Perch’s shared contracts deploy with no constructor arguments.

Each address is a function of the registry and the wasm hash.

Accounts pin them by hash, so no admin can repoint them.

Account upgrades wait about 7 days





ACCOUNTS

ACCOUNTS AND FACTORIES

Rule 0, admin, covers calls on the account itself, including apply_doc: full owner authority.

Perch’s factory derives the address from the admin signers.

Nido’s factory reserves the address before the passkey exists.

WHAT AN APP CAN ASK NIDO FOR

CONNECT

The account’s C-address, and nothing else.

SIGN ONE TRANSACTION

A passkey popup. Nido’s relayer submits and returns the hash. Calls on the account only.

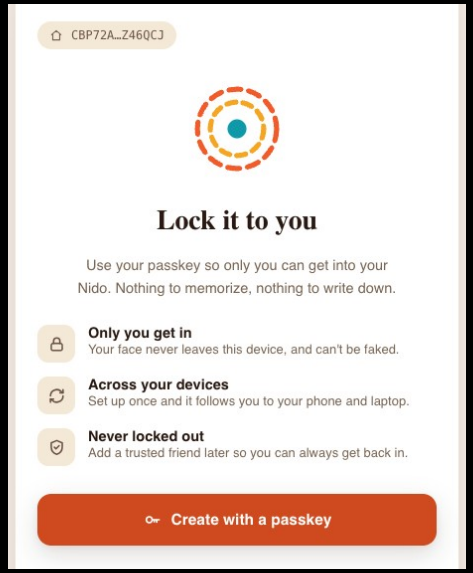
A SCOPED SESSION

The app holds a key; the owner approves a rule for it once.

NOT THE ACCOUNT ITSELF

Recovery, devices and policies stay in Nido’s own pages.

A PASSKEY ACCOUNT



test.nido.fyi on testnet

RESERVE

The C-address exists first

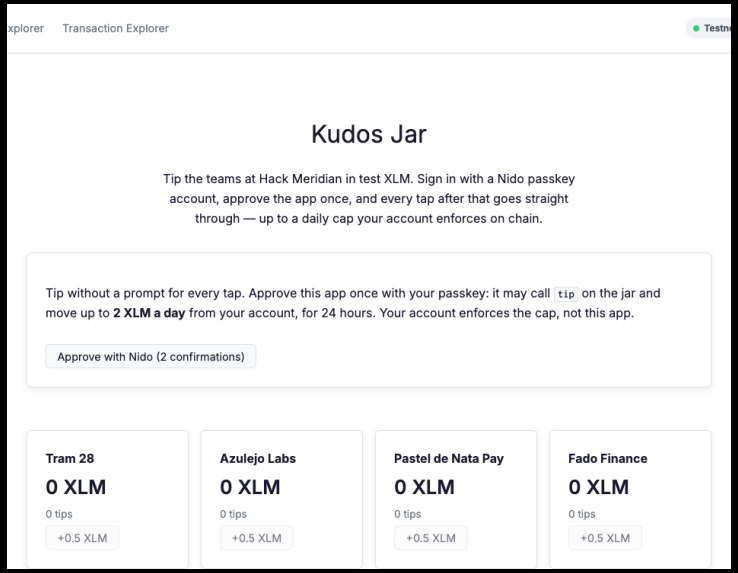
ONE PASSKEY

Bound to the account's subdomain

FUNDED

5 test XLM, fees sponsored

CONNECT, THEN ASK ONCE



Kudos Jar, running locally

PICK NIDO

From the usual wallet list

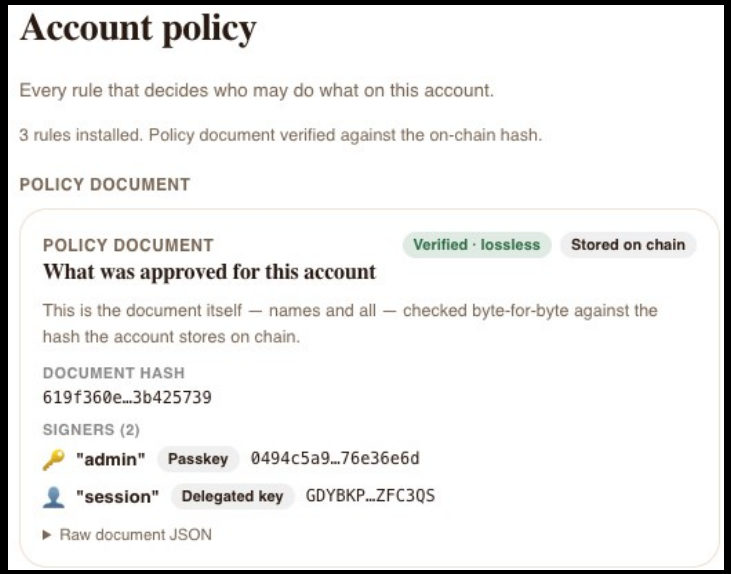
THE ACCOUNT

C-address and its XLM balance

ONE REQUEST

Two rules, two confirmations

THE ACCOUNT'S POLICY



Nido's policy page

DOCUMENT

Hash checked against the chain

RULES

admin, kudos-tip, kudos-xlm

KEYS

A passkey and the app's key

KUDOS JAR'S CALL SITE

TypeScript

```
const tx = await kudosJar.tip(
  { from: address, team: team.id, amount: 5_000_000n },
  { publicKey: transactionSource(address) },
)
const { result } = await submitContract(tx, address)
```

transactionSource and submitContract come from the template's app-lib. submitCall now does both in one call.

WHAT THE AGENT TRIPPED ON

A SILENT OLD TOOL

A global Scaffold too old for the config compiled contracts and wrote no clients. The template now installs its own.

A BLOCKED POPUP

The app simulated before opening Nido’s popup, and the browser forgot the click. The popup is now reserved during the click.

THE WRONG PATH, TWICE

First popup signing, then a contract that skipped require_auth. AGENTS.md now says both.

FOUND ALONG THE WAY

NIDO'S SIGNER

Signs under the admin rule only, so it can't authorize calls into apps.

NIDO'S RELAYER

Can't relay CAP-71 entries yet, so session keys pay their own fees.

NIDO'S DOCS

Still say the wallet returns signed XDR; it now submits and returns a hash.

FROM THE DRY RUN

Say why the relayer refused, and let the app pay its own fee. The template's half is ready.